

实验三：网络聊天室

实验内容

建立云服务器，在云服务器上使用Nginx进行适当配置，然后使用Javascript、Nodejs、Websocket编程，实现网络聊天室。

实验环境

腾讯云轻量应用服务器。操作系统 Ubuntu Server 24.04 LTS 64bit.

实验步骤

开放端口

在腾讯云控制台防火墙界面添加规则，开放端口 3000.

安装 node.js 与 Nginx

在控制台输入

```
sudo apt install nodejs npm nginx
```

编写 WebSocket 代码 server.js

```
const http = require('http');
const WebSocket = require('ws');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
  const file = req.url === '/' ? '/index.html' : req.url;
  fs.readFile(path.join(__dirname, file), (err, data) => {
    if (err) {
      res.writeHead(404);
      res.end("Not found");
    } else {
      res.writeHead(200);
      res.end(data);
    }
  });
});

const wss = new WebSocket.Server({ server });
```

```

wss.on('connection', ws => {
  ws.on('message', msg => {
    const messageStr = msg.toString();

    let user = "匿名";
    let text = messageStr;

    try {
      const parsed = JSON.parse(messageStr);
      user = parsed.user || "匿名";
      text = parsed.text || "";
    } catch (e) {
    }

    const broadcastMsg = `[${user}]: ${text}`;

    // 广播给所有客户端
    wss.clients.forEach(client => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(broadcastMsg);
      }
    });
  });
});

server.listen(3000, () => {
  console.log("Server is listening on port 3000");
});

```

编写前端界面 index.html

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>网络聊天室</title>
  <style>
    #username-section {
      margin-bottom: 10px;
    }
    #msg {
      width: 300px;
    }
  </style>
</head>
<body>
  <h2>聊天室</h2>

```

```
<div id="username-section">
  <input id="username-input" placeholder="设置用户名" />
  <button onclick="setUsername()">设置用户名</button>
  <span id="current-username">当前用户名: 匿名</span>
</div>

<input id="msg" placeholder="输入消息" disabled />
<button onclick="sendMsg()" disabled id="send-btn">发送</button>

<ul id="chat"></ul>

<script>
  let username = "匿名";

  const usernameInput = document.getElementById('username-input');
  const currentUsernameSpan = document.getElementById('current-username');
  const msgInput = document.getElementById('msg');
  const sendBtn = document.getElementById('send-btn');

  const ws = new WebSocket('ws://' + location.host);

  ws.onmessage = (event) => {
    const li = document.createElement('li');
    li.innerText = event.data;
    document.getElementById('chat').appendChild(li);
  };

  function setUsername() {
    const name = usernameInput.value.trim();
    if (name === "") {
      alert("用户名不能为空!");
      return;
    }
    username = name;
    currentUsernameSpan.innerText = "当前用户名: " + username;
    usernameInput.value = "";
    msgInput.disabled = false;
    sendBtn.disabled = false;
    msgInput.focus();
  }

  function sendMsg() {
    const message = msgInput.value.trim();
    if (!message) return;

    const msgObj = {
      user: username,
      text: message
    };
  };
</script>
```

```
        ws.send(JSON.stringify(msgObj));
        msgInput.value = '';
    }
</script>
</body>
</html>
```

使用 Nginx 做反向代理

编辑 Nginx 配置文件 /etc/nginx/sites-available/default：

```
server {
    listen 80;

    server_name 139.155.108.193;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

编辑后重启 Nginx：

```
sudo systemctl restart nginx
```

启动服务端

到 server.js 所在目录下，启动

```
node server.js
```

打开聊天室

在浏览器中输入云服务器的公网 IP，即可进入聊天室。

实验截图

聊天室截图如下

聊天室

当前用户名：张三

- [张三]: 你们好，我是张三
- [李四]: 你好啊，我是李四
- [王二]: 你好，我是王二

聊天室

当前用户名：李四

- [张三]: 你们好，我是张三
- [李四]: 你好啊，我是李四
- [王二]: 你好，我是王二

聊天室

当前用户名：王二

- [张三]: 你们好，我是张三
- [李四]: 你好啊，我是李四
- [王二]: 你好，我是王二